

MARAN PMS — Prompt 52: Refactor Arquitectura (Incremental)

Objetivo

Reorganizar `server/routes.ts` por dominios, agregar seguridad de producción, y extender la auditoría existente. Sin reescribir lógica existente. Sin romper nada.

Principio

Cada cambio es independiente. Si uno falla, los demás siguen funcionando. Orden recomendado: primero seguridad, luego auditoría, luego reorganización de rutas.

PARTE 1 — SEGURIDAD DE PRODUCCIÓN

1.1 Agregar helmet y rate limiting

En `server/index.ts` (o donde se inicializa Express), agregar **antes** de cualquier ruta:

```
import helmet from "helmet";
import rateLimit from "express-rate-limit";

// Helmet: headers de seguridad HTTP
app.use(helmet({
  contentSecurityPolicy: false, // desactivar CSP para no romper el frontend React
  crossOriginEmbedderPolicy: false,
}));

// Rate limit general para todas las rutas API
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos
  max: 500, // máximo 500 requests por IP por ventana
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: "Demasiadas solicitudes, intente en unos minutos" },
  skip: (req) => req.path.startsWith("/api/public/"), // no limitar web-checkin p
});
app.use("/api", apiLimiter);
```

```
// Rate limit estricto para login (anti-brute-force)
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 10, // solo 10 intentos de login por IP cada 15 minutos
  message: { error: "Demasiados intentos de inicio de sesión" },
});
app.use("/api/auth/login", loginLimiter);
```

Instalar si no están:

```
npm install helmet express-rate-limit
```

1.2 Deshabilitar /api/auth/setup en producción

En `server/routes.ts`, el endpoint `POST /api/auth/setup` debe estar completamente deshabilitado en producción:

```
app.post("/api/auth/setup", async (req, res) => {
  // BLOQUEAR EN PRODUCCIÓN
  if (process.env.NODE_ENV === "production") {
    return res.status(404).json({ error: "Not found" });
  }
  // ... resto del código existente sin cambios
});
```

1.3 SESSION_SECRET sin fallback hardcodeado

En la configuración de sesión (en `server/index.ts`), encontrar donde dice:

```
secret: process.env.SESSION_SECRET || "algún-valor-hardcodeado"
```

Reemplazar por:

```
const sessionSecret = process.env.SESSION_SECRET;
if (!sessionSecret && process.env.NODE_ENV === "production") {
  throw new Error("SESSION_SECRET es requerido en producción");
}
// En desarrollo se permite un fallback
secret: sessionSecret || "dev-secret-only-for-local",
```

1.4 Ocultar /api/source en producción

El endpoint que expone el código fuente (`GET /api/source/files`) solo debe estar disponible para admins y no en producción:

```
app.get("/api/source/files", requireAuth, async (req, res) => {
  if (process.env.NODE_ENV === "production") {
    return res.status(404).json({ error: "Not found" });
  }
  // ... resto del código existente
});
```

En el frontend (`client/src/components/app-sidebar.tsx`), ocultar el ítem de “Código Fuente” si `NODE_ENV === "production"` o si el usuario no es admin:

```
// Solo mostrar si es admin Y no es producción
{currentUser?.role === "admin" && import.meta.env.DEV && (
  <SidebarMenuItem>
    {/* ítem de código fuente */}
  </SidebarMenuItem>
)}
```

PARTE 2 — AUDITORÍA EXTENDIDA

Contexto

La tabla `audit_logs` ya existe con campos: `userId`, `userName`, `action`, `module`, `entityType`, `entityId`, `description`, `details`, `ipAddress`, `timestamp`.

Las acciones definidas son: `"create"` | `"update"` | `"delete"` | `"login"` | `"logout"` | `"view"` | `"export"`.

`createAuditLog` ya existe en storage pero **casi no se usa** — solo hay 1 llamada real en todo el sistema.

2.1 Helper de auditoría reutilizable

En `server/routes.ts` (o en un nuevo archivo `server/audit.ts`), crear una función helper que simplifique el registro:

```
// server/audit.ts
import { storage } from "../storage";

export async function audit(
```

```

    req: any,
    action: "create" | "update" | "delete" | "login" | "logout" | "export",
    module: string,
    description: string,
    options?: {
      entityType?: string;
      entityId?: string;
      details?: Record<string, any>;
    }
  ) {
    try {
      await storage.createAuditLog({
        userId: req.user?.id || null,
        userName: req.user?.fullName || req.user?.username || "Sistema",
        action,
        module,
        entityType: options?.entityType || null,
        entityId: options?.entityId || null,
        description,
        details: options?.details ? JSON.stringify(options.details) : null,
        ipAddress: req.ip || req.connection?.remoteAddress || null,
        timestamp: new Date(),
      });
    } catch (e) {
      // Nunca fallar por un error de auditoría
      console.error("Audit log error:", e);
    }
  }
}

```

2.2 Registrar eventos críticos que hoy no tienen auditoría

Agregar llamadas a `audit()` en los siguientes endpoints. Usar el helper del punto anterior — una línea por evento:

Login/Logout (ya hay endpoints, agregar audit):

```

// En POST /api/auth/login, dentro del callback de éxito:
await audit(req, "login", "auth", `Inicio de sesión: ${user.username}`);

// En POST /api/auth/logout:
await audit(req, "logout", "auth", `Cierre de sesión: ${req.user?.username}`);

```

Reservas (cambios de estado críticos):

```

// En PATCH /api/reservations/:id — después de updateReservation exitoso:

```

```

await audit(req, "update", "reservations",
  `Reserva ${existing.reservationCode} modificada`,
  { entityType: "reservation", entityId: req.params.id, details: cambios }
);

// En POST /api/reservations/:id/check-in:
await audit(req, "update", "reservations",
  `Check-in: ${reservation.reservationCode} - Hab. ${room.roomNumber}`,
  { entityType: "reservation", entityId: req.params.id }
);

// En POST /api/reservations/:id/check-out:
await audit(req, "update", "reservations",
  `Check-out: ${reservation.reservationCode} - Hab. ${room.roomNumber}`,
  { entityType: "reservation", entityId: req.params.id }
);

// En PATCH /api/reservations/:id/cancel (o equivalente):
await audit(req, "delete", "reservations",
  `Cancelación: ${reservation.reservationCode}`,
  { entityType: "reservation", entityId: req.params.id }
);

```

Pagos (todos los pagos son críticos):

```

// En POST /api/payments:
await audit(req, "create", "payments",
  `Pago registrado: $$${req.body.amount} (${req.body.method}) - Reserva ${req.body}`
  { entityType: "payment", entityId: created.id }
);

```

Tarifas (cambios de precio):

```

// En PATCH /api/rate-plans/:id:
await audit(req, "update", "rate-plans",
  `Tarifa modificada: ${existing.name} - $$${existing.baseRate} → $$${req.body.base}`
  { entityType: "rate_plan", entityId: req.params.id }
);

// En POST /api/rate-plans:
await audit(req, "create", "rate-plans",
  `Nueva tarifa creada: ${req.body.name}`,
  { entityType: "rate_plan", entityId: created.id }
);

```

Grupos (pagos y check-out grupal):

```
// En POST /api/groups/:id/payment:
await audit(req, "create", "groups",
  `Pago grupal: ${req.body.amount} (${req.body.method}) - Grupo ${group.name}`,
  { entityType: "group", entityId: req.params.groupId }
);

// En POST /api/groups/:id/check-out-all:
await audit(req, "update", "groups",
  `Check-out grupal: ${result.processed} habitaciones - ${group.name}`,
  { entityType: "group", entityId: req.params.groupId }
);
```

Cajas (apertura y cierre de turno):

```
// En POST /api/cash/shifts (apertura):
await audit(req, "create", "cash",
  `Turno de caja abierto - Área: ${req.body.area}`,
  { entityType: "cash_shift", entityId: created.id }
);

// En POST /api/cash/shifts/:id/close:
await audit(req, "update", "cash",
  `Turno de caja cerrado - Área: ${shift.area}`,
  { entityType: "cash_shift", entityId: req.params.id }
);
```

Configuración del sistema (cambios administrativos):

```
// En PATCH /api/system-settings/:id:
await audit(req, "update", "admin",
  `Configuración modificada: ${existing.key}`,
  { entityType: "system_setting", entityId: req.params.id }
);
```

PARTE 3 — REORGANIZACIÓN DE ROUTES.TS POR DOMINIOS

Contexto

server/routes.ts tiene ~42.000 líneas. Ya hay 4 archivos separados que funcionan:

- `server/exports.ts` → `registerExportRoutes(app)`
- `server/adminCash.ts` → `registerAdminCashRoutes(app)`
- `server/billing/routes.ts` → `registerBillingRoutes(app)`
- `server/reports/routes.ts` → `registerReportsRoutes(app)`

El patrón ya existe y funciona. Hay que aplicarlo al resto.

Estructura objetivo

```
server/
  routes/
    auth.ts           ← login, logout, setup, me
    reservations.ts   ← reservas, check-in, check-out, charges, payments
    planning.ts       ← planning, calendar
    rooms.ts          ← rooms, room-types, bed-types, housekeeping
    guests.ts         ← guests, companies, agencies
    groups.ts         ← groups, group-blocks, group-payments
    restaurant.ts     ← restaurant (ya es 48 endpoints)
    spa.ts            ← spa (ya es 43 endpoints)
    events.ts         ← events (ya es 37 endpoints)
    hospitality.ts    ← hospitality, stay-notes, alerts
    inventory.ts      ← inventory (ya está parcialmente separado)
    ota.ts            ← ota-channels, ota-reservations, webhook
    packages.ts       ← packages, package-items
    admin.ts          ← system-users, settings, audit-logs, notifications
    routes.ts         ← solo importa y registra todos los módulos
```

Cómo hacerlo sin romper nada

Metodología para cada módulo (hacer de a uno):

1. Crear el archivo nuevo `server/routes/[modulo].ts`
2. Copiar los endpoints del módulo desde `routes.ts` al nuevo archivo
3. Envolver en una función `export function register[Modulo]Routes(app: Express) {
... }`
4. Al inicio del archivo, copiar los imports necesarios (storage, db, requireAuth, etc.)
5. En `routes.ts`, reemplazar los endpoints copiados por una sola línea:
`register[Modulo]Routes(app)`

6. Verificar que el servidor siga funcionando

7. Recién entonces pasar al siguiente módulo

Empezar por los módulos más independientes (menos dependencias cruzadas):

Orden recomendado:

1. `hospitality.ts` — 9 endpoints, muy independiente
2. `ota.ts` — 6 endpoints + webhook
3. `rooms.ts` — 6 endpoints de rooms básicos
4. `guests.ts` — 13 endpoints de guests
5. `inventory.ts` — 15 endpoints (ya está parcialmente separado)
6. `planning.ts` — 1 endpoint, muy simple
7. `groups.ts` — 18 endpoints
8. `restaurant.ts` — 48 endpoints (el más grande después de spa)
9. `spa.ts` — 43 endpoints
10. `events.ts` — 37 endpoints
11. `reservations.ts` — 24 endpoints (dejar para el final por ser el más usado)

Template para cada archivo nuevo:

```
// server/routes/hospitality.ts
import type { Express } from "express";
import { storage } from "../storage";
import { requireAuth } from "../auth"; // o como esté importado
import { db } from "../db";

export function registerHospitalityRoutes(app: Express) {
  // Pegar aquí todos los app.get/post/patch/delete de hospitality
  // sin ningún cambio en la lógica interna
}
```

En `routes.ts` al final del proceso:

```
// server/routes.ts — versión final limpia
import { registerHospitalityRoutes } from "../routes/hospitality";
import { registerOtaRoutes } from "../routes/ota";
```



```
// ... etc

export async function registerRoutes(httpServer: Server, app: Express) {
  // Middleware de auth (mantener aquí)
  // Endpoints de auth (mantener aquí o mover a routes/auth.ts)

  // Módulos
  registerHospitalityRoutes(app);
  registerOtaRoutes(app);
  registerRoomsRoutes(app);
  registerGuestsRoutes(app);
  registerInventoryRoutes(app);
  registerPlanningRoutes(app);
  registerGroupsRoutes(app);
  registerRestaurantRoutes(app);
  registerSpaRoutes(app);
  registerEventsRoutes(app);
  registerReservationsRoutes(app);

  // Ya existentes
  registerExportRoutes(app);
  registerAdminCashRoutes(app);
  registerBillingRoutes(app);
  registerReportsRoutes(app);
}
```

NOTAS TÉCNICAS

- **Instalar dependencias** de la Parte 1 antes de implementar: `npm install helmet express-rate-limit`
- **El helper de audit** nunca debe romper el flujo principal — siempre envuelto en `try/catch` que solo loguea el error
- **La reorganización de rutas** es puro movimiento de código, sin cambios de lógica. Si algo deja de funcionar es por un `import` faltante, no por lógica rota
- **Orden de implementación recomendado:** Parte 1 (seguridad) → Parte 2 (auditoría) → Parte 3 (reorganización) — cada una es independiente
- **NODE_ENV** en Replit: verificar cómo está configurado en producción. Si no se usa `NODE_ENV=production` **explícitamente**, ajustar las condiciones usando otra variable de entorno (ej: `REPLIT_DEPLOYMENT=true`)

- La tabla `audit_logs` ya tiene índices implícitos en `id` . Si el volumen de logs crece, agregar índice en `module` y `timestamp` para las queries de la página de administración